

Anaconda与Python第一份实验

第002单元 练习详解

以下解释与001使用同一组原始数据。先写自己的答案，再核对。输出相同只是核对的一部分，还应解释每个值什么时候产生，哪些变化会让原结论不再成立。

一 变量追踪

开始时这些名字还没有值。 $x = 3$ 后， x 为3； $bias = 1$ 后， $bias$ 为1； $prediction = 2 * x + bias$ 先取3和1算出7，因此 $prediction$ 为7。执行 $x = 4$ 之后， x 变为4， $bias$ 仍为1， $prediction$ 仍为7。

赋值保存这次计算的值，不建立自动更新的电子表格公式。再次执行 $prediction = 2 * x + bias$ ，才能得到9。如果你重跑一个显示 $prediction$ 的单元格，它只显示已有值，不会替你重跑计算单元格。

$x = x + 1$ 合法，因为先读取旧值，再写入新值。数学等式 $x=x+1$ 在通常实数范围内没有解，两者含义不同。判断相等应使用 $==$ ，例如 $prediction == 7$ 产生布尔值，不改变 $prediction$ 。

二 类型与运算

$2 * 3$ 得到整数6，类型`int`。 $2 * "3"$ 得到字符串"33"，类型`str`，是把一个字符串重复两次。 $1 / 5$ 得到浮点数0.2，类型`float`。 $7 == 8$ 得到`False`，类型`bool`。

"3" + 1不能直接把文本与整数相加；如果任务确实要求数值相加，可先用`int("3")`转换得到3，再加1得到4。不要把“程序不再报错”当成转换正确性的证明。原始字段若是编号003，转整数会丢掉前导零，其任务意义可能已经改变。

$3 == 3.0$ 得到`True`，但`type(3)`和`type(3.0)`不同。本讲接受4与4.0的数值一致，不要求所有输出字符逐字相同。对不易精确表示的小数，不能把任意两条数学等价表达式都用精确相等判断；第014单元会继续讨论。

三 列表与索引

`train_x[2]`是3，`train_y[2]`是8，因为索引0对应第一条，索引1对应第二条，索引2对应第三条。长度为5的列表，最后一个合法非负索引为4。索引5试图访问第六条，产生`IndexError`。

只检查长度不够。比如把`train_y`改成`[3, 5, 9, 8, 11]`，长度仍为5，却把第三与第四行标签交换。代码可以正常算出分数，但已经不是001的数据。正确的数据核对必须检查对应关系，不只是行数。

`backup = train_y`不会自动创建独立列表。若随后对`backup`的某个元素赋新值，可能同时改变`train_y`所指向的同一个列表。本讲变化实验重新建立完整的新标签列表，保持原数据可核对。列表复制与对象身份在后续单元细讲。

四 逐行累计误差

索引	输入	标签	A在偏移量1时预测	绝对误差	本轮结束累计值
0	1	3	3	0	0
1	2	5	5	0	0
2	3	8	7	1	1
3	4	9	9	0	1
4	5	11	11	0	1

总误差是1，数量为5，平均为0.2。第三条低估1，后面两条正确并不会抹去它。循环里的`total = total + error`使用上轮累计值，而不是每次重新从0开始。

若把`total = 0`放进循环，累计过程会变成“仅记住本轮误差”，最后一条误差为0，最终错误输出0.0。修复方法是把初始化移到循环之前，保持最后除法在循环之后。直接打印0.2不能修复算法，因为换数据之后仍然会错。

用标签[8, 8]、预测[7, 9]核对另一种常见错误：有符号误差为-1和1，相加抵消为0；绝对误差为1和1，MAE为1。只有在逐条相减之后取绝对值，才能保留两次偏差的大小。

五 候选选择与同分

偏移量	预测列表	绝对误差列表	总误差	MAE
0	2, 4, 6, 8, 10	1, 1, 2, 1, 1	6	1.2
1	3, 5, 7, 9, 11	0, 0, 1, 0, 0	1	0.2
2	4, 6, 8, 10, 12	1, 1, 0, 1, 1	4	0.8

先用候选0初始化最好分数为1.2。外层再次遇到0时，它不严格更好，保留原值；遇到1时，0.2小于1.2，于是更新；遇到2时，0.8不小于0.2，不更新。最终选中1。

同分时取较小偏移量，需要两个条件共同保证：候选按升序排列；只在`score < best_score`时更新。若写成`<=`，后来的同分候选会替换前面一个，可能变成较大的偏移量。若候选不排序，严格小于只保证保留先遇到者，不自动保证数值最小。

本实验选择中只有`train_x`、`train_y`和候选进入计算。`test_y`没有参与。若先看测试分数再改候选集合，或者在测试上不断修改B的未命中输出，评价协议已经变了，不能继续声称是原来的封存检查。

六 查表函数的两次查询

查询3时，第0轮比较 $3 == 1$ 为False；第1轮 $3 == 2$ 为False；第2轮 $3 == 3$ 为True，于是返回`known_y[2]`，即8。`return`立即结束本次函数调用，第四、第五条无需继续查。

查询2.5时，与1、2、3、4、5都不相等，循环全部结束，才执行外侧`return 0`。这个0是事先规定的未命中输出，不是从测试标签里估计出来的值。

如果`return 0`误放在`for`内部、但在`if`之外，查询3在第一条不匹配后就返回0，永远到不了第三条。修复时把`return 0`移到与`for`相同的缩进层级。不要删除第一条训练记录来制造第一轮命中。

函数的训练列表必须等长、对齐，训练输入应唯一。给定数据满足这些前提。若真实应用有重复输入但标签冲突，还需制定规则；本单元函数不回答那些超出声明范围的问题。

七 训练 测试与压力测试

B在五条训练输入上逐一返回原标签，训练MAE为0。A用偏移量1时训练MAE为0.2。这是真实的训练比较，应保留。

测试输入	标签	A预测	B预测	A绝对误差	B绝对误差
1.5	4	4	0	0	4
2.5	6	6	0	0	6
3.5	8	8	0	0	8
4.5	10	10	0	0	10

A测试总误差0，MAE为0。B总误差28，MAE为 $28/4=7$ 。这四个输入均未在训练表出现，所以B的输出全来自预定的未命中分支。

压力标签按 $3x+1$ 构造，依次是5.5、8.5、11.5、14.5。A仍输出4、6、8、10，绝对误差依次是1.5、2.5、3.5、4.5，总和12，平均3。这里没有重新训练A，目的就是观察原规则在改变关系后的表现。

一种合格结论是：“在给定五条合成训练记录上，三个候选中的偏移量1使规则A的MAE最小，为0.2。查表规则B的训练MAE为0，但在四个指定的未见输入上，其MAE为7，A为0。关系改成 $3x+1$ 时，冻结的A的MAE升至3。公开答案练习只能说明这些机制，不能证明真实任务的普遍表现。”

八 六个错误示例的最小修复

`01_name_error.py`先引用不存在的`not_defined_bias`，产生`NameError`。应在使用前给需要的名字赋值，或修正误拼的名字。不能依赖另一个临时窗口曾经存在的变量。

`02_type_error.py`把"3"与1相加，产生`TypeError`。若目标是数值4，把"3"转换为整数或直接使用数值3；若目标是文本31，应把1也明确写成字符串。要先确定任务，而不是随机转换。

03_index_error.py读取五项列表的索引5，产生IndexError。读取最后一项可用索引4。本题不要求通过随意追加数据让索引5变得合法，那会改变输入。

04_indent_error.py在if下一行漏掉缩进，产生IndentationError。把print缩进四格，表明它属于条件成立时执行的块。代码缩进应由算法结构决定，不应为了消除报错而随处添加空格。

05_reset_total.py不会报错，却输出0.0。把total = 0移出循环，再运行同一数据，得到0.2。还应换一组末尾不是最大误差的数据核对，防止程序只在某个特殊例子上凑巧正确。

06_early_return.py的return在第一轮后直接退出，输出0.0而不是0.2。把return移到for外侧，仍在def内部。通过打印每轮索引可发现原程序只处理了索引0；修复后应处理0到4。

以上六种问题中，前四种被解释器明确拒绝，后两种需要你用计算含义发现。软件安装完成并不能消除算法错误；反过来，一次NameError也不证明安装失败。

九 两个变化实验

把原训练第三条8改为7，得到[3, 5, 7, 9, 11]。候选0每条低估1，MAE为1；候选1全部正确，MAE为0；候选2每条高估1，MAE为1。仍选1，但A训练误差变为0。这个改变应另存为新实验，不冒充原数据的结果。

把训练标签换成[2.5, 4.5, 6.5, 8.5, 10.5]时，候选0每条低估0.5，MAE为0.5；候选1每条高估0.5，MAE同为0.5；候选2每条高估1.5，MAE为1.5。升序加严格小于保留候选0。这个例子主动测试了原数据未触发的同分分支。

进一步将候选顺序改为[1, 0, 2]，原样的严格小于逻辑会保留1。因此讲义明确把候选升序作为本实现的前提。003会把这类前提变成更系统的输入检查与测试，而不是靠运行者默默记住。

十 Notebook与复现诊断

第一格赋bias = 1，第二格打印2 * 3 + bias，得到7。把第一格文字改成2但不执行，内核仍保存1，所以第二格仍打印7。先运行第一格再运行第二格，才得到8。

清除输出不清除变量；重启内核会清除旧状态。重启后直接执行第二格，bias未定义，产生NameError。正确完整Notebook要从第一格开始运行，把依赖重新建立起来。仅凭屏幕上单元格顺序整齐，不能证明实际运行顺序整齐。

脚本与Notebook的数值结果应一致，但环境记录可能显示不同Python版本。如果它们实际使用不同解释器，应先记录并校正环境，再讨论是否复现。同样的文件、数据和明确执行顺序是必要条件；对更复杂实验，随机状态、数值库和硬件还会影响结果。本讲没有随机数，也没有数值库，刻意先控制这些因素。